(LINEAR SEARCH)

```
import time
import matplotlib.pyplot as plt
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
n = [1000, 2000, 3000, 4000]
t = []
for size in n:
    arr = list(range(size))
    key = size - 1
    start = time.time()
    linear_search(arr, key)
    end = time.time()

    t.append(end - start)
plt.plot(n, t)
plt.xlabel("Array Size")
plt.ylabel("Time")
plt.title("Linear Search Time Complexity")
plt.show()
```

(BINARY SEARCH)

```
import time
import matplotlib.pyplot as plt
def binary_search(a, x):
    l, h = 0, len(a)-1
    while l <= h:
        m = (l+h)//2
        if a[m] == x:
            return m
        elif a[m] < x:
            l = m + 1
        else:
            h = m - 1
n = [2000, 5000, 10000]
t = []
for i in n:
    a = list(range(i))
    s = time.time()
    binary_search(a, i//2)
    t.append(time.time() - s)
plt.plot(n, t)
plt.xlabel("Array Size")
plt.ylabel("Time")
plt.title("Binary Search Time Complexity")
plt.show()
```

(TOWER OF HANOI)

```
def hanoi(n, s, a, d):
    if n == 1:
        print("Move disk 1 from", s, "to", d)
        return
    hanoi(n-1, s, d, a)
    print("Move disk", n, "from", s, "to", d)
    hanoi(n-1, a, s, d)

n = int(input("Enter number of disks: "))
hanoi(n, 'A', 'B', 'C')
```

(SELECTION SORT)

```
import time
import random
import matplotlib.pyplot as plt
def selection_sort(a):
    for i in range(len(a)):
        m = i
        for j in range(i+1, len(a)):
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
n = [100, 500, 1000]
t = []
for i in n:
    a = random.sample(range(1000), i)
    s = time.time()
    selection_sort(a)
    t.append(time.time() - s)
plt.plot(n, t)
plt.xlabel("Number of Elements")
plt.ylabel("Time Taken")
plt.title("Selection Sort Time Complexity")
plt.show()
```

```
(DIVIDE & CONQ)
def minmax(arr):
    if len(arr) == 1:
        return arr[0], arr[0]

    mid = len(arr) // 2
    min1, max1 = minmax(arr[:mid])
    min2, max2 = minmax(arr[mid:])

    return min(min1, min2), max(max1, max2)
arr = [100, 11, 445, 1, 330, 3000]
minimum, maximum = minmax(arr)
print("Minimum element is:", minimum)
print("Maximum element is:", maximum)
```

(QUICK SORT)

```
import time
import random
import matplotlib.pyplot as plt
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)
n = [100, 500, 1000, 2000, 5000]
t = []
for i in n:
    arr = [random.randint(1,1000) for j in range(i)]
    s = time.time()
    quick_sort(arr)
    e = time.time()
    t.append(e - s)
plt.plot(n, t)
plt.xlabel("Number of Elements")
plt.ylabel("Time Taken (s)")
plt.title("Quick Sort Time Complexity")
plt.show()
```

(BINARY SEARCH)

```
def optimal_bst(freq):
    n = len(freq)
    cost = [[0]*n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for l in range(2, n+1):
        for i in range(n-l+1):
            j = i+l-1
   cost[i][j] = sum(freq[i:j+1]) + min(cost[i+1][j], cost[i][j-1])

    return cost[0][n-1]
freq = [34, 8, 50]
print("Optimal BST Cost:", optimal_bst(freq))
```

```
(FLOYDS)
INF = 999
G = [
   [0, 5, INF, 10],
   [INF, 0, 10, 5],
   [30, INF, 0, 15],
   [15, INF, 5, 0]
]
n = 4
for k in range(n):
   for i in range(n):
      for j in range(n):
 G[i][j] = min(G[i][j], G[i][k] + G[k][j])
for i in range(n):
   for j in range(n):
      if G[i][j] == INF:
         print("INF", end="\t")
      else:
         print(G[i][j], end="\t")
   print()
```

(BOYER-MOORE)

```
def boyer_moore(text, pattern):
    n = len(text)
    m = len(pattern)
    i = 0
while i <= n - m:
j = m - 1
while j >= 0 and pattern[j] == text[i + j]:
j -= 1
if j < 0:
print("Pattern found at shift", i)
return
else:
i += 1
text = "ABAAABCD"
pattern = "ABC"
boyer_moore(text, pattern)
```

(BFS TRAVERSAL)

```
from collections import deque
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}
visited = []
queue = deque(['A'])
while queue:
   node = queue.popleft()
   if node not in visited:
      print(node, end=" ")
      visited.append(node)
      queue.extend(graph[node])
```

(PRIMS ALGORITM)

```
import heapq
graph = {
   'A': [('B',1), ('C',2)],
   'B': [('A',1), ('D',3)],
   'C': [('A',2)],
   'D': [('B',3)]
}
visited = set()
heap = [(0, 'A')]
mst = []
while heap:
   w, node = heapq.heappop(heap)
   if node not in visited:
      visited.add(node)
      mst.append((w, node))
      for n, wt in graph[node]:
         if n not in visited:
      heapq.heappush(heap, (wt, n))
print(mst)
```

(KRUSHKALS)

```
def find(p, x):
    while p[x] != x:
        x = p[x]
    return x
edges = [(0,1,10), (0,2,6), (0,3,5), (1,3,15), (2,3,4)]
edges.sort(key=lambda x: x[2])
p = [0,1,2,3]
print("Minimum Spanning Tree:")
for u, v, w in edges:
    x = find(p, u)
    y = find(p, v)
    if x != y:
        print(u, "-", v, "=", w)
        p[x] = y
```

```
(SUBSET)
def subset(a, t, s=[]):
    if t == 0:
        return [s]
    if not a or t < 0:
        return []
    return subset(a[1:], t-a[0], s+[a[0]]) + subset(a[1:], t, s)
a = [1,2,5,6,8]
r = subset(a, 9)
print("The subsets of", a, "that sum to 9 are")
print(r[0], "and", r[1])
```